PROJETO PRÁTICO
ANÁLISE DE ALGORITMOS

Documento de orientação, desenvolvimento, entregas e avaliação

Implementação • Experimentação • Análise Empírica • Análise Assintótica • Comparação de Algoritmos

Disciplina: Análise de Algoritmos

Projeto com acompanhamento parcial nas aulas práticas e relatório técnico na modalidade EAD

1. Apresentação do projeto

O Projeto Prático de Análise de Algoritmos tem como finalidade conduzir a investigação do comportamento de algoritmos por meio da implementação, execução, medição e comparação de diferentes soluções. O trabalho deverá aproximar a análise teórica das decisões observadas na prática profissional de Computação, permitindo compreender como o tamanho e a organização das entradas influenciam o tempo de processamento e o uso de recursos.

O foco do projeto não está apenas em fazer o programa funcionar. O aluno deverá demonstrar que compreende o algoritmo utilizado, identifica suas operações relevantes, planeja experimentos, registra resultados, interpreta evidências e consegue justificar tecnicamente a escolha de uma solução. O documento dá enfase para a linguagem C/C++, porém a escolha da linguagem de programação é livre.

2. Objetivos

- Implementar e executar algoritmos em C/C++.
- Identificar as operações primitivas ou operações dominantes de cada algoritmo.
- Variar o tamanho das entradas e observar o impacto no tempo de processamento.
- Investigar como a organização dos dados de entrada interfere no comportamento do algoritmo.
- Organizar os resultados em tabelas e, quando adequado, gráficos de desempenho.
- Comparar a análise empírica com a análise assintótica.
- Analisar melhor caso, pior caso e caso médio quando esses casos forem aplicáveis ao algoritmo estudado.
- Comparar estratégias e algoritmos diferentes para um mesmo problema ou para problemas equivalentes.
- Justificar decisões considerando corretude, simplicidade, eficiência, consumo de recursos e adequação ao contexto.

3. Organização do projeto e formas de entrega

O projeto será desenvolvido de forma progressiva. As entregas e análises ocorrerão em dois momentos complementares:

Momento	Objetivo	Evidência esperada
Aulas práticas de segunda-feira	Acompanhamento parcial do desenvolvimento, testes, análise e comparação dos algoritmos.	Mini seminário com demonstração do projeto, resultados parciais e decisões tomadas.
Análise de Algoritmos - EAD	Consolidação técnica do trabalho desenvolvido.	Relatório técnico contendo implementações, telas de execução, testes, tabelas/gráficos, análise crítica e conclusões.

Importante: as apresentações parciais não substituem o relatório técnico. Elas fazem parte do processo de acompanhamento e deverão mostrar a evolução real do projeto.

4. Conteúdos e algoritmos que poderão compor o projeto

Ao longo das etapas, o projeto deverá utilizar os conteúdos trabalhados na disciplina. Dependendo da proposta definida para a turma, poderão ser investigados:

- Busca sequencial e busca binária;
- estruturas iterativas;
- algoritmos recursivos;
- Quick Sort;
- Merge Sort;
- estratégias de divisão e conquista;
- algoritmos gulosos;
- programação dinâmica.

Quando houver duas ou mais soluções para o mesmo problema, a comparação deverá ser explícita. Não basta apresentar os códigos separadamente: é necessário confrontar comportamento, custo, vantagens, limitações e situações em que cada solução é mais adequada.

5. Metodologia mínima obrigatória para cada experimento

Para cada algoritmo ou comparação realizada, o aluno deverá seguir o roteiro abaixo.

1. Definir o problema - Explicar claramente o que o algoritmo deve resolver e quais são as entradas e saídas.

2. Apresentar o algoritmo - Informar a estratégia utilizada e mostrar a implementação em C/C++.

3. Identificar as operações relevantes - Indicar comparações, atribuições, acessos, trocas, somas, multiplicações, chamadas recursivas ou outras primitivas que representem o custo do algoritmo.

4. Definir os tamanhos de entrada - Escolher diferentes valores de n, começando por entradas menores e aumentando progressivamente até que seja possível observar tendência de crescimento.

5. Definir os formatos de entrada - Preparar conjuntos de dados diferentes, adequados ao algoritmo estudado.

6. Executar os testes - Realizar as execuções nas mesmas condições, registrar tempos e demais medidas de interesse.

7. Organizar os resultados - Apresentar os dados em tabelas e, quando adequado, em gráficos.

8. Fazer a análise assintótica - Apresentar a função de custo ou recorrência e indicar a ordem de crescimento.

9. Confrontar teoria e prática - Comparar os resultados medidos com o comportamento esperado teoricamente.

10. Concluir tecnicamente - Justificar qual solução é mais adequada ao problema e em quais condições.

6. Tipos de testes obrigatórios/sugeridos

Os testes não deverão utilizar apenas um vetor aleatório. O formato dos dados de entrada deverá ser planejado de acordo com o algoritmo analisado, porque diferentes organizações dos dados podem produzir comportamentos distintos.

Tipo de entrada	Exemplo	Finalidade da análise
Aleatória	Valores em ordem não previsível.	Observar um comportamento próximo de situações gerais e comparar execuções.
Ordenada crescente	[1, 2, 3, 4, 5, ...]	Investigar efeitos de entradas favoráveis ou desfavoráveis a determinados algoritmos.
Ordenada decrescente	[100, 99, 98, ...]	Explorar possíveis situações de pior comportamento, quando aplicável.
Quase ordenada	Poucos elementos fora de posição.	Avaliar sensibilidade do algoritmo à organização inicial dos dados.
Com valores repetidos	[4, 7, 4, 9, 7, ...]	Verificar comportamento diante de duplicidades.
Busca - elemento no início/meio/fim/ausente	Chave em posições diferentes ou inexistente.	Comparar melhor, pior e caso médio quando aplicável.

7. Registro da análise empírica

Cada experimento deverá registrar informações suficientes para permitir comparação. Recomenda-se repetir as execuções mais de uma vez quando o tempo for muito pequeno ou houver variações perceptíveis entre medições.

- Tamanho da entrada (n).
- Formato/organização da entrada.
- Tempo de execução.
- Quantidade de comparações, trocas, chamadas recursivas ou outra operação dominante, quando for possível instrumentar o código.
- Observações relevantes sobre comportamento, erros, limitações ou diferenças entre execuções.

Modelo de tabela para os resultados:

Algoritmo	n	Formato da entrada	Tempo	Operação dominante	Quantidade/observação
Ex.: Quick Sort	1.000	Aleatória	...	Comparações	...

8. Análise teórica obrigatória

A análise empírica deverá ser acompanhada de análise teórica. Para cada algoritmo, apresentar, quando aplicável:

- operações primitivas ou operação dominante;
- função de custo ou recorrência;
- termo dominante;
- complexidade assintótica;
- melhor caso, pior caso e caso médio;
- complexidade de espaço, quando ela for relevante para a comparação.

Não será suficiente escrever apenas "O(n)", "O(n log n)" ou "O(n^2)". O relatório deverá mostrar como essa classificação foi obtida a partir da estrutura do algoritmo.

9. Tabelas, gráficos e interpretação

Os gráficos deverão ser usados para apoiar a interpretação dos resultados, e não apenas como elemento visual. Sempre que forem incluídos, deverão apresentar título, identificação dos eixos e unidade de medida.

- Eixo horizontal: tamanho da entrada (n).
- Eixo vertical: tempo, quantidade de operações ou outra medida analisada.
- Quando houver comparação, representar as diferentes soluções no mesmo contexto de medição.
- Após cada tabela ou gráfico, escrever uma interpretação: o que cresceu, quanto cresceu, qual algoritmo se destacou e se o resultado é coerente com a análise assintótica.

10. Análise crítica esperada

A análise crítica deverá responder, com base nos resultados obtidos, a questões como:

- Qual algoritmo apresentou melhor desempenho para entradas pequenas? E para entradas maiores?
- A organização inicial do vetor alterou o comportamento observado? Por quê?
- O resultado empírico confirmou a ordem de crescimento prevista teoricamente?
- Houve casos em que um algoritmo assintoticamente melhor não foi o mais rápido? Que fatores podem explicar isso?
- Qual solução é mais simples de implementar e manter?
- Há aumento significativo no uso de memória?

- A recursividade trouxe benefícios ou custos adicionais?
- A solução é correta para todos os tipos de entrada testados?
- Em qual contexto prático cada algoritmo seria recomendado?

11. Entregas parciais e mini seminários - aulas práticas de segunda-feira

Durante as aulas práticas, cada grupo/aluno deverá apresentar a evolução do projeto. A apresentação deverá ser objetiva e demonstrar trabalho efetivamente realizado.

Etapa	O que apresentar	Evidências mínimas
Etapa 1 - Planejamento	Problema, algoritmos selecionados, hipóteses e plano de testes.	Descrição do problema e definição das entradas.
Etapa 2 - Implementação	Códigos em funcionamento e primeiras execuções.	Trechos relevantes do código e telas de execução.
Etapa 3 - Experimentação	Resultados parciais para diferentes tamanhos e formatos de entrada.	Tabela inicial de medições e identificação das operações relevantes.
Etapa 4 - Análise	Comparação empírica e assintótica.	Gráficos, função de custo/recorrência e interpretação.
Etapa final - Mini seminário	Síntese do projeto e conclusão técnica.	Demonstração, resultados principais e justificativa da solução recomendada.

12. Relatório técnico obrigatório - Análise de Algoritmos EAD

O relatório técnico deverá consolidar o desenvolvimento do projeto. A estrutura mínima obrigatória é a seguinte:

1. Identificação do projeto: Título, integrantes, turma e tema/problema investigado.

2. Introdução: Contextualização do problema, objetivo do trabalho e justificativa dos algoritmos escolhidos.

3. Descrição das soluções: Explicação dos algoritmos e estratégias implementadas.

4. Implementação: Trechos relevantes do código C/C++. Não é necessário preencher o relatório com páginas de código sem explicação.

5. Planejamento dos testes: Tamanhos de entrada, formatos dos vetores/dados, quantidade de execuções e critérios de medição.

6. Evidências de execução: Telas legíveis mostrando o programa funcionando e resultados obtidos.

7. Resultados experimentais: Tabelas com tempos, operações e demais medidas coletadas.

8. Gráficos de desempenho: Representação visual dos resultados quando adequada.

9. Análise assintótica: Primitivas, função de custo/recorrência, termo dominante e ordem de crescimento.

10. Análise crítica e comparação: Interpretação dos resultados, diferenças entre teoria e prática, vantagens e limitações.

11. Conclusão: Síntese do aprendizado e justificativa da solução recomendada para o contexto estudado.

12. Referências: Livros, materiais, sites e demais fontes realmente utilizados.

Sobre as telas de execução: as imagens devem estar legíveis, recortadas de forma adequada e acompanhadas de legenda ou explicação do que está sendo demonstrado. Uma imagem sem análise não substitui a interpretação técnica.

13. O que deve aparecer em cada comparação de algoritmos

Critério	Pergunta orientadora
Corretude	A solução produz o resultado esperado para os diferentes testes?
Simplicidade	Qual solução é mais simples de compreender, implementar e manter?
Eficiência temporal	Como o tempo/quantidade de operações cresce com n?
Eficiência espacial	Há diferença relevante no uso de memória?
Sensibilidade à entrada	Dados ordenados, reversos, aleatórios ou repetidos alteram o comportamento?
Escalabilidade	O que ocorre quando o tamanho da entrada aumenta significativamente?
Adequação ao contexto	Em que situação prática a solução deve ser escolhida?

14. Critérios de avaliação

A avaliação considerará o processo de desenvolvimento e a qualidade da análise, e não apenas o funcionamento do programa.

Critério	Descrição
Prática de Análise de Algorimtos: Implementação e corretude Mini seminários e acompanhamento	Programas executam corretamente e tratam adequadamente as entradas propostas. Clareza, domínio do que foi desenvolvido e evolução demonstrada nas entregas parciais.
Análise de Algoritmos – EaD: Documentação e relatório técnico Planejamento e execução dos testes	Variedade de tamanhos e formatos de entrada, coerência dos testes e registro das condições.

Análise empírica Análise assintótica Análise crítica e comparação Organização do relatório técnico	Medições, tabelas, gráficos e identificação das operações relevantes. Função de custo/recorrência, termo dominante e classificação justificada. Interpretação dos resultados, confronto teoria x prática e justificativa técnica. Estrutura, clareza, evidências, referências e qualidade da apresentação.

15. Uso de fontes e ferramentas de IA

Ferramentas de apoio, inclusive inteligência artificial, podem ser utilizadas para esclarecer conceitos, revisar código ou apoiar a organização do trabalho, desde que o aluno mantenha autoria e domínio sobre o que entrega.

- Todo código apresentado deve ser compreendido pelo aluno e explicado quando solicitado.
- Resultados gerados por ferramentas devem ser verificados por execução e análise própria.
- Não serão aceitos textos, gráficos, códigos ou conclusões incluídos sem validação.
- As fontes utilizadas devem ser registradas nas referências do relatório.
- Justificar as escolhas feitas no projeto durante as apresentações.

16. Resultado esperado

Ao final do projeto, espera-se que o aluno seja capaz de ir além da implementação. Deverá demonstrar, com evidências, como um algoritmo se comporta, explicar por que esse comportamento ocorre, relacionar os resultados medidos à análise assintótica e justificar tecnicamente qual solução é mais adequada para um determinado contexto.

Implementar → Medir → Comparar → Analisar → Justificar